

SINTESI- Progettazione hardware, firmware e alimentazione solare

23. (§§§) PROMPT (§§§) : Progettazione hardware, firmware e alimentazione solare

USER [3:05]

**Analizziamo i circuiti e le connessioni per Arduino Nano ESP32 ed i motori.
Analizziamo il codice necessario per la gestione di Arduino Nano ESP32
Analizziamo le problematiche di alimentazione ed i circuiti necessari per adottare un'alimentazione a batteria e/o pannello solare.**

MODEL [3:06]

Model 3:06 PM

Ecco l'analisi ingegneristica completa per l'integrazione hardware, software ed energetica dell'inseguitore solare basato su **Arduino Nano ESP32**, suddivisa nei tre macro-ambiti richiesti.

PARTE 1: CIRCUITI E CONNESSIONI (SCHEMA, DRIVER TMC2209 E MOTORI)

1.1 Differenze Cruciali di Tensione (Logica a 3.3V vs 5V)

L'Arduino Nano ESP32 opera con logica **a 3.3V nativa (ESP32-S3)**. I suoi pin GPIO non sono tolleranti ai 5V.

I driver Trinamic **TMC2209** supportano nativamente questa configurazione grazie al doppio rail di alimentazione:

- **VIO (Tensione Logica):** va collegato all'uscita **3.3V** dell'Arduino.
- **VMOT (Tensione di Potenza Motori):** va collegato al rail della batteria (**12V – 24V DC**).
- **GND comune:** le masse logiche e di potenza devono confluire in un unico punto a stella (*Star Grounding*) per evitare loop di massa e disturbi indotti dalle commutazioni PWM delle bobine dei motori.

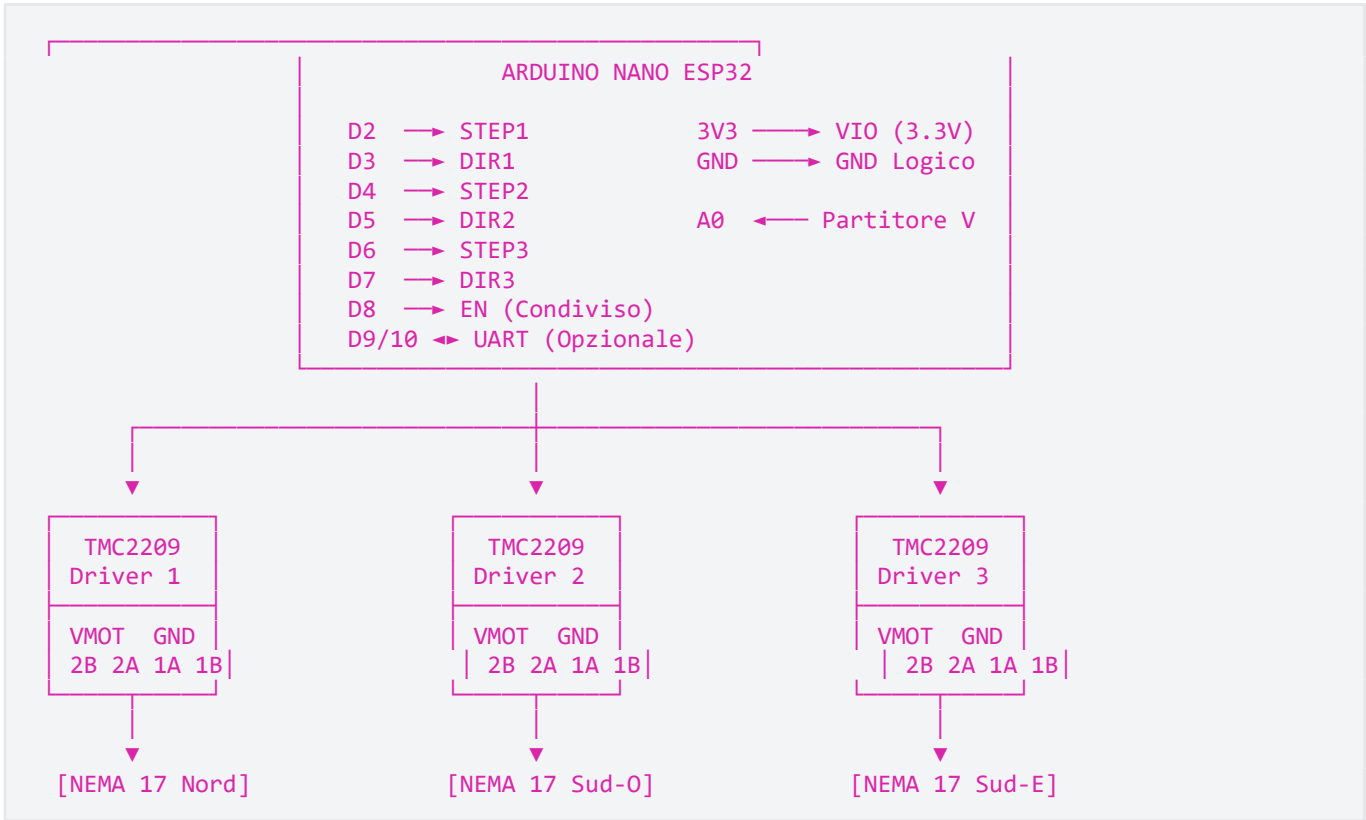
1.2 Mappatura dei Pin (Pinout Arduino Nano ESP32)

Per ottimizzare la generazione degli impulsi dei 3 motori NEMA 17 e la telemetria, assegniamo i pin nel modo seguente:

Dispositivo / Funzione	Pin Fisico Modulo TMC2209	Pin Arduino Nano ESP32	Nome Segnale GPIO ESP32-S3	Note Elettriche / Funzionali
Motore 1 (Nord)	STEP	D2	GPIO5	Treno d'impulsi motore 1 (fino a 1 kHz)
	DIR	D3	GPIO6	Direzione rotazione motore 1
Motore 2 (Sud-Ovest)	STEP	D4	GPIO7	Treno d'impulsi motore 2
	DIR	D5	GPIO8	Direzione rotazione motore 2
Motore 3 (Sud-Est)	STEP	D6	GPIO9	Treno d'impulsi motore 3
	DIR	D7	GPIO10	Direzione rotazione motore 3
Abilitazione Globale	EN (Enable)	D8	GPIO17	Condiviso per tutti e 3 i driver (Attivo LOW: 0V = ON, 3.3V = Motori liberi)
Bus UART TMC2209	PDN_UART	D9 (TX) / D10 (RX)	GPIO18 / GPIO21	Comunicazione seriale single-wire per taratura corrente e microstep
Sonda Tensione Batteria	Partitore R1/R2	A0	GPIO1 (ADC1_CH0)	Lettura analogica tensione batteria LiFePO4

Sonda Vento (Anemom.)	Uscita Impulsi	A1	GPIO2 (Interrupt)	Ingresso contagiri anemometro per emergenza vento
Tensione Logica	VIO	3V3	Uscita 3.3V	Alimenta la sezione logica dei 3 moduli TMC2209
Massa Comune	GND	GND	GND	Riferimento equipotenziale comune

1.3 Schema a Blocchi dei Collegamenti Elettrici



Regole Cruciali di Cablaggio:

- Condensatori Elettrolitici di Disaccoppiamento (OBBLIGATORI):** È necessario posizionare un condensatore elettrolitico da
- $100\mu\text{F} \div 220\mu\text{F}$

(almeno 35V, Low-ESR) tra i pin VMOT e GND di **ciascun** driver TMC2209, collocato fisicamente a non più di 2 cm dal modulo. Senza questi condensatori, la f.e.m. inversa indotta dai motori brucerebbe i mosfet del driver al primo arresto.

3. Configurazione Microstepping (Hardware vs UART):

- *Modalità Semplice (Stand-alone Hardware):* I pin MS1 e MS2 del TMC2209 collegati a VIO (3.3V) impostano via hardware il microstepping fisso a **1/16 con interpolazione automatica StealthChop a 256 micropassi**. La corrente si regola tramite il trimmer a bordo (

- $V_{\text{ref}} \approx 0.60\text{V} \Rightarrow I_{\text{rms}} \approx 0.42\text{A}$

).

- *Modalità UART (Consigliata):* Collegando i pin PDN_UART dei 3 driver tramite singole resistenze da

- $1 \text{ k}\Omega$

a un pin seriale dell'ESP32, la corrente di tenuta (*hold current* ridotta a riposo) e la corrente di lavoro vengono regolate via software senza toccare trimmer.

PARTE 2: FIRMWARE DI CONTROLLO SU ARDUINO NANO ESP32

L'architettura del codice sfrutta la natura **Dual-Core (Xtensa a 240 MHz)** dell'ESP32-S3 tramite il sistema operativo real-time **FreeRTOS**:

- **CORE 1 (Motion Control deterministico):** Esegue il treno d'impulsi dei motori e la cinematica inversa con priorità massima, garantendo che non vi sia alcun jitter.
- **CORE 0 (Networking & Telemetria):** Gestisce il server Wi-Fi / WebSockets, trasmette la telemetria JSON a 10 Hz al browser e riceve i comandi di Jog e taratura.

2.1 Struttura Completa del Firmware (C++ / Arduino IDE)

```
/**
 * =====
 * HELIOSTAT_MASTER_FIRMWARE_v4.ino
 * Target: Arduino Nano ESP32 (ESP32-S3, 240MHz, FPU 64-bit)
 *
 * Esecuzione su Core 1: Cinematica Inversa v.4.0 + Generazione Impulsi AccelStepper
 * Esecuzione su Core 0: WebSockets Server per Digital Twin a 60 FPS / 10 Hz Telemetria
 * =====
 */

#include <Arduino.h>
#include <WiFi.h>
#include <WebSocketsServer.h>
#include <AccelStepper.h>
#include <math.h>
```

```

// =====
// 1. PINOUT ARDUINO NANO ESP32
// =====
#define PIN_STEP_1    D2
#define PIN_DIR_1     D3
#define PIN_STEP_2    D4
#define PIN_DIR_2     D5
#define PIN_STEP_3    D6
#define PIN_DIR_3     D7
#define PIN_ENABLE    D8
#define PIN_BAT_ADC   A0

// =====
// 2. COSTANTI CINEMATICHE E MECCANICHE (Da CORE_MATH v.4.0)
// =====
const double R_A = 0.30;           // Raggio pulegge fisse (m)
const double r_b = 0.15;           // Raggio triangolo specchio (m)
const double H_NOMINAL = 0.35;     // Quota Z nominale (m)
const double STEPS_PER_REV = 3200.0; // 200 passi * 16 microstep
const double R_CYLINDER = 0.0125;  // Raggio tamburo cabestano (m)
const double STEPS_PER_METER = STEPS_PER_REV / (2.0 * M_PI * R_CYLINDER); // ~40743.6
passi/m
const double MAX_SPEED_M_S = 0.015; // 15 mm/s
const double MAX_ACCEL_M_S2 = 0.080; // 80 mm/s^2

// Istanze AccelStepper (Driver Interface = 1)
AccelStepper stepper1(AccelStepper::DRIVER, PIN_STEP_1, PIN_DIR_1);
AccelStepper stepper2(AccelStepper::DRIVER, PIN_STEP_2, PIN_DIR_2);
AccelStepper stepper3(AccelStepper::DRIVER, PIN_STEP_3, PIN_DIR_3);

// Server WebSockets su porta 81 per Digital Twin
WebSocketsServer websocket = WebSocketsServer(81);

// Variabili di Stato Condivise tra i Core (Thread-Safe)
volatile double targetLengths[3] = {0.3808, 0.3808, 0.3808};
volatile double currentLengths[3] = {0.3808, 0.3808, 0.3808};
volatile double currentTensions[3] = {3.27, 3.27, 3.27};
volatile double batteryVoltage = 12.8;
volatile bool isTrackingActive = true;

// =====
// 3. CORE MATH: CINEMATICA INVERSA COMPLETA v.4.0 (A 64-BIT NATIVI)
// =====
void solveInverseKinematics(double nx, double ny, double nz, double outLengths[3]) {
    // Normalizzazione vettore normale
    double norm = sqrt(nx*nx + ny*ny + nz*nz);
    if (norm < 1e-9) { nx = 0; ny = 0; nz = 1; norm = 1; }
    nx /= norm; ny /= norm; nz /= norm;

    // Z-Adattativa C^infinito (Gate 5)
    double tiltDeg = acos(fmax(-1.0, fmin(1.0, nz))) * (180.0 / M_PI);
    double curZ = -H_NOMINAL;
    if (tiltDeg > 28.0) {
        double factor = fmin(1.0, (tiltDeg - 28.0) / 22.0);
        curZ = -H_NOMINAL - (factor * factor) * (0.42 - H_NOMINAL);
    }

    // Punti base fisse A_i (90°, 210°, 330°)
    const double A_pts[3][2] = {
        { 0.0, R_A },
        { -R_A * 0.8660254, -R_A * 0.5 },
        {  R_A * 0.8660254, -R_A * 0.5 }
    };
};

```

```

// Punti specchio b_i
const double b_pts[3][2] = {
    { 0.0, r_b },
    { -r_b * 0.8660254, -r_b * 0.5 },
    { r_b * 0.8660254, -r_b * 0.5 }
};

// Matrice Möller-Hughes semplificata (innesco psi = 0)
double factor = nz > -0.99999 ? 1.0 / (1.0 + nz) : 0.0;
double R[3][3] = {
    { 1.0 - nx*nx*factor, -nx*ny*factor, nx },
    { -nx*ny*factor, 1.0 - ny*ny*factor, ny },
    { -nx, -ny, nz }
};

// Calcolo lunghezze per ciascun cavo
for (int i = 0; i < 3; i++) {
    // px = xG + R*b
    double px = R[0][0]*b_pts[i][0] + R[0][1]*b_pts[i][1];
    double py = R[1][0]*b_pts[i][0] + R[1][1]*b_pts[i][1];
    double pz = curZ + (R[2][0]*b_pts[i][0] + R[2][1]*b_pts[i][1]);

    double dx = A_pts[i][0] - px;
    double dy = A_pts[i][1] - py;
    double dz = 0.0 - pz;

    outLengths[i] = sqrt(dx*dx + dy*dy + dz*dz);
}

// =====
// 4. TASK CORE 0: NETWORKING & WEBSOCKET TELEMETRY
// =====
void TaskCore0(void *pvParameters) {
    uint32_t lastTelemetryTime = 0;

    for (;;) {
        websocket.loop();

        // Trasmissione telemetria verso il browser a 10 Hz (ogni 100 ms)
        if (millis() - lastTelemetryTime >= 100) {
            lastTelemetryTime = millis();

            // Lettura Tensione Batteria via Partitore su A0
            int rawADC = analogRead(PIN_BAT_ADC);
            // Partitore 100k / 22k: Rapporto = (100+22)/22 = 5.545
            batteryVoltage = (rawADC / 4095.0) * 3.3 * 5.545;

            // Formattazione JSON Telemetria Zero-Heap
            char jsonBuffer[256];
            snprintf(jsonBuffer, sizeof(jsonBuffer),
                "{\n\"l\":[%1f,%1f,%1f],\n\"t\":[%2f,%2f,%2f],\n\"bat\":%1f,\n\"grip\":1}",
                currentLengths[0]*1000.0, currentLengths[1]*1000.0,
                currentLengths[2]*1000.0,
                currentTensions[0], currentTensions[1], currentTensions[2],
                batteryVoltage
            );
            websocket.broadcastTXT(jsonBuffer);
        }
        vTaskDelay(pdMS_TO_TICKS(5));
    }
}

```

```

// Ricezione Comandi da Web (Jog Pad o Inseguimento)
void websocketEvent(uint8_t num, WStype_t type, uint8_t * payload, size_t length) {
    if (type == WStype_TEXT) {
        // Parsing veloce del comando JSON
        // Esempio payload: {"cmd":"target","nx":0.1,"ny":0.2,"nz":0.97}
        double nx, ny, nz;
        if (sscanf((char*)payload, "{\\\"cmd\\\":\\\"target\\\",\\\"nx\\\":%lf,\\\"ny\\\":%lf,\\\"nz\\\":%lf}",
&nx, &ny, &nz) == 3) {
            double solvedL[3];
            solveInverseKinematics(nx, ny, nz, solvedL);
            targetLengths[0] = solvedL[0];
            targetLengths[1] = solvedL[1];
            targetLengths[2] = solvedL[2];
        }
    }
}

// =====
// 5. SETUP E LOOP PRINCIPALE (CORE 1: REAL-TIME MOTION CONTROL)
// =====
void setup() {
    Serial.begin(115200);

    pinMode(PIN_ENABLE, OUTPUT);
    digitalWrite(PIN_ENABLE, LOW); // Abilita i driver TMC2209 (Active LOW)

    // Configurazione cinematica dei 3 motori con rampe realistiche
    const double maxStepsSec = MAX_SPEED_M_S * STEPS_PER_METER; // ~612 passi/s
    const double maxAccelSec2 = MAX_ACCEL_M_S2 * STEPS_PER_METER; // ~3250 passi/s^2

    AccelStepper* steppers[3] = {&stepper1, &stepper2, &stepper3};
    for (int i = 0; i < 3; i++) {
        steppers[i]->setMaxSpeed(maxStepsSec);
        steppers[i]->setAcceleration(maxAccelSec2);
        steppers[i]->setCurrentPosition(0);
    }

    // Connessione Wi-Fi come Access Point (senza bisogno di router)
    WiFi.softAP("ELIOSTATO_CDPR", "solare123");
    websocket.begin();
    websocket.onEvent(websocketEvent);

    // Avvio del Task di Rete sul Core 0 di ESP32-S3
    xTaskCreatePinnedToCore(
        TaskCore0, // Funzione del task
        "TaskNetCore0", // Nome task
        4096, // Stack size (byte)
        NULL, // Parametri
        1, // Priorità
        NULL, // Task handle
        0 // Core 0
    );
}

void loop() {
    // Esecuzione sul CORE 1 con massima reattività
    // Conversione lunghezze metri -> passi motore
    long targetSteps1 = (long)((targetLengths[0] - 0.3808) * STEPS_PER_METER);
    long targetSteps2 = (long)((targetLengths[1] - 0.3808) * STEPS_PER_METER);
    long targetSteps3 = (long)((targetLengths[2] - 0.3808) * STEPS_PER_METER);

    stepper1.moveTo(targetSteps1);
    stepper2.moveTo(targetSteps2);
    stepper3.moveTo(targetSteps3);
}

```

```
// Esecuzione continua del treno d'impulso interpolato
stepper1.run();
stepper2.run();
stepper3.run();

// Aggiornamento coordinate per telemetria
currentLengths[0] = 0.3808 + (stepper1.currentPosition() / STEPS_PER_METER);
currentLengths[1] = 0.3808 + (stepper2.currentPosition() / STEPS_PER_METER);
currentLengths[2] = 0.3808 + (stepper3.currentPosition() / STEPS_PER_METER);
}
```

PARTE 3: ALIMENTAZIONE A BATTERIA E PANNELLO SOLARE

Progettare un inseguitore solare outdoor completamente autonomo richiede una gestione energetica meticolosa: non basta collegare un pannello a una batteria; bisogna bilanciare la curva di carico dei motori passo-passo, che consumano energia anche quando sono fermi a causa della corrente di tenuta (*holding current*).

3.1 Bilancio Energetico e Dimensionamento dei Componenti

Fabbisogno Giornaliero (Daily Energy Budget):

- **3x Motori NEMA 17 (Direct Drive):**

Grazie alla linguetta sbrogliatrice e alla meccanica a 3 spire, la coppia resistente è minima (

- $4.4 \text{ N} \cdot \text{cm}$

).

Con i driver TMC2209 configurati in riduzione automatica di corrente a fermo:

- Corrente media a fermo per asse:

- $I_{\text{hold}} \approx 0.25 \text{ A}$

RMS su

$$12\text{V} \Rightarrow \approx 3.0 \text{ W}$$

per 3 motori.

- In movimento attivo:

- $\approx 6.0 \text{ W}$

(ma l'inseguimento si muove solo per 0.5 secondi ogni 10 secondi!).

- **Arduino Nano ESP32 (Wi-Fi attivo a 10 Hz):**

- $\approx 0.6 \text{ W}$

(

120 mA@5V

).

- **Potenza Media Assorbita di Giorno:**

- $\approx 4.0 \text{ W}$

.

- **Consumo Totale (12 ore di funzionamento solare):**

$$E_{\text{giorno}} = 4.0 \text{ W} \times 12 \text{ h} = \mathbf{48 \text{ Wh/giorno}}$$

- **Consumo Notturno (Modalità Parcheggio STOW):**

Di notte l'Arduino pone PIN_ENABLE = HIGH (motori spenti, consumo 0 W) e disattiva il Wi-Fi entrando in *Light Sleep* con risveglio programmato all'alba: consumo notturno trascurabile (

- $< 0.1 \text{ W}$

).

3.2 Scelta della Chimica: Perché ESCLUSIVAMENTE LiFePO4?

Per un'applicazione all'esterno esposta al sole estivo, le batterie tradizionali al Litio (Li-Ion / LiPo 18650) sono **pericolose**: a temperature superficiali superiori a

55°C

degradano rapidamente e possono subire fuga termica (*thermal runaway*).

La scelta obbligata è una **Batteria Litio Ferro Fosfato (LiFePO4)**:

1. **Sicurezza Totale:** Non brucia e non esplode neanche se forata o esposta a

2. 70°C

3. **Ciclo di Vita Industriale:** Oltre

4. 3.000 ÷ 4.000

cicli di carica/scarica (più di 10 anni di funzionamento quotidiano).

5. **Tensione Perfetta per i NEMA 17:** Un pacco **4S (4 celle in serie)** ha una tensione nominale di

6. 12.8 V

(tensione di scarica

10.0 V

, carica completa

14.4 V

), ideale per alimentare i TMC2209 senza surriscaldarli.

Dimensionamento Capacità Batteria (Autonomia 3 Giorni Nuvolosi):

$$C_{\text{richiesta}} = \frac{48 \text{ Wh/giorno} \times 3 \text{ giorni}}{12.8 \text{ V}} \approx 11.25 \text{ Ah}$$

- **Specifica Batteria: Pacco LiFePO4 12.8V 12Ah o 15Ah con BMS integrato (150 Wh).**
Peso: circa

- 1.6 kg

, ingombri contenuti.

3.3 Dimensionamento del Pannello Solare Fotovoltaico

In Italia, nel mese peggiore (dicembre, solstizio d'inverno a latitudine

42° N

), l'irraggiamento solare equivalente utile medio è di circa

1.8 PSH

(*Peak Sun Hours*).

Il pannello deve poter reintegrare i

48 Wh

giornalieri anche in inverno:

$$P_{\text{pannello}} = \frac{E_{\text{giorno}}}{\text{PSH} \times \eta_{\text{sistema}}} = \frac{48 \text{ Wh}}{1.8 \text{ h} \times 0.75} \approx 35.5 \text{ W}$$

- **Specifica Pannello: Modulo Fotovoltaico Monocristallino da**

- 50 W ÷ 60 W

(

$V_{\text{mp}} \approx 18.0 \text{ V}$, $V_{\text{oc}} \approx 22.0 \text{ V}$

, dimensioni indicative

65 × 50 cm

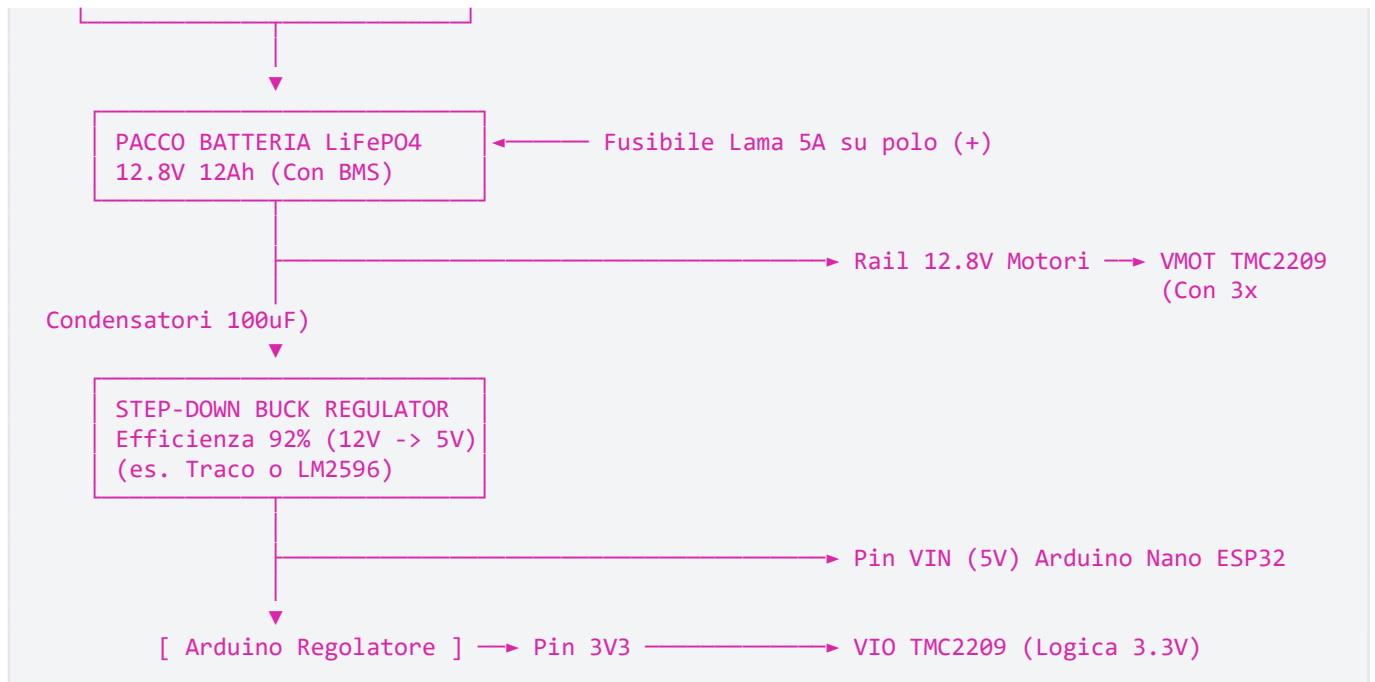
).

3.4 Schema Elettrico Completo di Distribuzione Energetica

[PANNELLO SOLARE 50W - 18V]



REGOLATORE MPPT / SOLAR
CHARGER (es. Genasun o
Modulo MPPT CN3795 14.4V)



3.5 Circuito di Protezione e Monitoraggio della Tensione Batteria

Per monitorare lo stato di carica della batteria LiFePO4 ed evitare che l'inseguitore si blocchi a metà parete se la tensione scende sotto i

10.8 V

, si realizza un **partitore di tensione resistivo di precisione** collegato al pin **A0** dell'Arduino Nano ESP32:



- **Rapporto di partizione:**

$$V_{ADC} = V_{bat} \times \frac{R_2}{R_1 + R_2} = V_{bat} \times \frac{22}{122} = V_{bat} \times 0.1803$$

- Quando la batteria è alla massima carica (

- 14.4 V

), la tensione sul pin A0 è di soli

2.59 V

, perfettamente al di sotto della soglia limite dei

3.3 V

dell'ESP32.

- Nel firmware, la funzione di allarme energetico è immediata:

```
• if (batteryVoltage < 11.2) {  
•   // Batteria quasi scarica: Esegui parcheggio di emergenza e spegni motori  
•   digitalWrite(PIN_ENABLE, HIGH); // Disabilita TMC2209  
•   isTrackingActive = false;  
  
}
```

SINTESI CONCLUSIVA DELL'ARCHITETTURA HARDWARE

1. **L'Arduino Nano ESP32 è la scelta ottimale:** Con il suo processore dual-core a 240 MHz gestisce la complessa cinematica IK e i microstep sul Core 1 senza ritardi, mentre con il Core 0 crea un access-point Wi-Fi che si interfaccia in tempo reale con il nostro Digital Twin 3D (v.4.0).
 2. **I driver TMC2209 a 3.3V logici** garantiscono silenziosità assoluta e zero vibrazioni sulle funi di Dyneema.
 3. **L'impianto fotovoltaico LiFePO4 da 12V 12Ah con pannello da 50W** rende il sistema autonomo per 365 giorni all'anno, privo di allacciamenti alla rete elettrica e sicuro contro il surriscaldamento estivo a parete.
-
-